FIRST & FOLLOW Sets — Advanced Practice

Compilers · Lecture 4: Parsers · Top-down / predictive parsing

Name: Zineb Elansari **Group:** none **Date:** 07/10/2026

Exercise 1 — G1

(a) Nullable: **A, C, S** (S → C → ε).

(b)

	FIRST	FOLLOW
S	{a, b, c, d, ε}	{b, $}
A	{a, ε}	{a, b, c, d}
C	{c, d, ε}	{b, $}

(c) In S → A S b, both A and S are nullable. So S ⇒ A S b ⇒ S b ⇒ C b ⇒ b, using A → ε, then S → C, then C → ε. The string “b” is derived from S, so b ∈ FIRST(S).

Exercise 2 — G2

(a) Nullable: A, B. C is not nullable, so S is not nullable.

	FIRST	FOLLOW
S	{a, b, c, x, z}	{$}
A	{a, b, c, x, z, ε}	{b, c, z}
B	{b, c, z, ε}	{c, x, z}
C	{c, z}	{y, $}

(b)

- FIRST(B x A) = {b, c, x, z}
- FIRST(A B) = {a, b, c, x, z, ε}
- FIRST(C y B) = {c, z}

(c) After A comes B C. B is nullable, but C is not, so something (c or z) always follows A and the end of input can never come right after it. For $ to enter FOLLOW(A), B C would have to be nullable (for example by adding C → ε). Then FOLLOW(S) ⊆ FOLLOW(A) and $ ∈ FOLLOW(A). Alternatively, A could be placed at the end of a production of S.

Exercise 3 — G3

(a) Nullable: B only.

	FIRST
S	{x, q, z, w}
A	{x}
B	{y, ε}
C	{z, w}

(b) FOLLOW iteration (pass 3 changes nothing, so the algorithm stops there).

	Init.	Pass 1	Pass 2	Pass 3	Pass 4
FOLLOW(S)	{$}	{$}	{$}	{$}	
FOLLOW(A)	∅	{p, r, s}	{p, r, s}	{p, r, s}	

	Init.	Pass 1	Pass 2	Pass 3	Pass 4
FOLLOW(B)	∅	{p, r}	{p, r, s}	{p, r, s}	
FOLLOW(C)	∅	{p, r, s}	{p, r, s}	{p, r, s}	

(c) The edges are A → B (from A → xB), B → C (from B → yC) and C → A (from C → zA). These form a cycle A → B → C → A. Sets in a cycle of inclusions must all be equal, so FOLLOW(A) = FOLLOW(B) = FOLLOW(C) = {p, r, s}, however many passes are used. S has no edges because it never ends a production, so FOLLOW(S) = {$} stays separate.

Exercise 4 — G4 and G4′

(a) Nullable: A.

	FIRST	FOLLOW
S	{a, b, c}	{d, $}
A	{a, b, c, ε}	{a, c}

(b) Nullable: A, A′.

	FIRST	FOLLOW
S	{a, b, c}	{$}
A	{a, b, c, ε}	{a}
A′	{a, c, ε}	{a}

(c) The FIRST sets are the same: FIRST(S) = {a, b, c} and FIRST(A) = {a, b, c, ε}. The FOLLOW sets differ: FOLLOW(S) is {d, $} in G4 but {$} in G4′, and FOLLOW(A) is {a, c} in G4 but {a} in G4′. FIRST depends only on what the symbols can derive, so it is preserved. FOLLOW depends on where a symbol appears in the right-hand sides, and the elimination removed the occurrences “S d” and “A c”.

(d) G4′ is **not LL(1)**:

- **S → A a** and **S → b** collide on **b**, because b ∈ FIRST(A a) and b ∈ FIRST(b). This is a FIRST/FIRST conflict.
- **A′ → a d A′** and **A′ → ε** collide on **a**, because a ∈ FIRST(a d A′) and a ∈ FOLLOW(A′). This is a FIRST/FOLLOW conflict.

Exercise 5 — G5

(a) Left-factored grammar:

```
S  → if E then S S′ | while E do S | begin L end | a
S′ → else S | ε
L  → S L′
L′ → ; L | ε
E  → id | num
```

(b)

	FIRST	FOLLOW
S	{if, while, begin, a}	{else, ;, end, $}
S′	{else, ε}	{else, ;, end, $}
L	{if, while, begin, a}	{end}
L′	{;, ε}	{end}
E	{id, num}	{then, do}

(c) The offending pair is **S′ → else S** and **S′ → ε** on the terminal **else**. This is the dangling-else problem. S′ is nullable, and else ∈ FIRST(else S) and else ∈ FOLLOW(S′). So the condition "FIRST(α) ∩ FOLLOW(A) = ∅ when A is nullable" is violated (a FIRST/FOLLOW conflict).

Exercise 6 — Marking the student's work (G6)

Nullable: A, B.

	Student's answer	Correct ?	Correct set	Mistake
FIRST(S)	{a, b, c, d}	Wrong	{a, b, c, d, e}	Forgot e: A and B are nullable, so e from "A B e" is first
FOLLOW(S)	{$}	Correct		
FIRST(A)	{a, ε}	Correct		
FOLLOW(A)	{b, c, d, e, $}	Wrong	{b, c, d, e}	$ added wrongly: A is followed by B e, and B is followed by e, not by the end of S
FIRST(B)	{b, c, d}	Wrong	{b, c, d, ε}	Forgot ε: B → ε
FOLLOW(B)	{e, ε}	Wrong	{e}	ε never belongs to a FOLLOW set
FIRST(C)	{c, d}	Correct		
FOLLOW(C)	{a, f}	Wrong	{a, e, f}	Forgot e: in B → C A, A is nullable, so FOLLOW(B) = {e} flows into FOLLOW(C)

That is exactly 5 wrong entries.

Exercise 7 — True or false?

1. **False.** FOLLOW sets contain only terminals and $, never ε, by definition. No grammar has this.
2. **True.** If A → αBβ and β ⇒* ε, then anything that can follow A can also follow B.
3. **False.** Disjoint FIRST sets are necessary but not sufficient. LL(1) also requires that, when an alternative can derive ε, the FIRST sets of the other alternatives are disjoint from FOLLOW(A). Counter-example: S → A a, A → a | ε. FIRST(a) = {a} and FIRST(ε) = {ε} are disjoint, but A is nullable and a ∈ FOLLOW(A). On lookahead a the parser cannot choose between A → a and A → ε (the input "a" needs A → ε, the input "aa" needs A → a).
4. **False.** They can be computed, as in G4 in Exercise 4(a).
5. **False.** Take S → A x | B y, A → a, B → a. FIRST(A) = FIRST(B) = {a}, but FOLLOW(A) = {x} and FOLLOW(B) = {y}.
6. **False.** Take S → a S b | ε. Then FOLLOW(S) = {b, $}, because S appears inside its own production followed by b.

Exercise 8 — G8

(a) Nullable: A, B, C and S (all of them).

	FIRST	FOLLOW
S	{a, b, c, d, e, ε}	{$}
A	{a, b, c, d, ε}	{b, d, $}
B	{b, d, ε}	{c, $}
C	{d, ε}	{c, e, $}

(b) LL(1) parsing table (conflicting cells are highlighted).

	a	b	c	d	e	$
S	S→AB	S→AB	S→AB	S→AB S→Ce	S→Ce	S→AB
A	A→aA	A→Bc A→ε	A→Bc	A→Bc A→ε		A→ε
B		B→bB	B→C	B→C		B→C
C			C→ε	C→dC	C→ε	C→ε

(c) There are three multiply-defined cells:

- **[S, d]** is a FIRST/FIRST conflict. S → A B starts with d through A ⇒ ε and B ⇒ C ⇒ dC. S → C e also starts with d through C ⇒ dC.
- **[A, b]** is a FIRST/FOLLOW conflict. A → B c starts with b, and A → ε is chosen on FOLLOW(A), which contains b. This is because A is nullable and is followed by B, which can start with b.
- **[A, d]** is a FIRST/FOLLOW conflict for the same reason. d ∈ FIRST(B c) through B ⇒ C ⇒ dC, and d ∈ FOLLOW(A).

(d) The strings **“d”** and **“de”**. Both start with d. “d” needs S → A B (A ⇒ ε, B ⇒ C ⇒ d), while “de” needs S → C e. One lookahead symbol (d) cannot decide.

Exercise 9 — Design challenge

(a) Grammar:

```
S → A B
A → a A | ε
B → c S b | ε
```

(b)

	FIRST	FOLLOW
S	{a, c, ε}	{b, $}
A	{a, ε}	{b, c, $}
B	{c, ε}	{b, $}

Why these sets come out right:

- FIRST(S) = {a, c, ε}, because A and B are both nullable.
- FOLLOW(S) = {$, b}. The b comes from B → c S b.
- FOLLOW(A) = {c} ∪ FOLLOW(S) = {b, c, $}, because B follows A and B is nullable.
- FOLLOW(B) = FOLLOW(S) = {b, $}.

LL(1) check (only for non-terminals with more than one production):

- **A:** FIRST(aA) = {a}, and ε is chosen on FOLLOW(A) = {b, c, $}. These are disjoint, so there is no conflict.
- **B:** FIRST(cSb) = {c}, and ε is chosen on FOLLOW(B) = {b, $}. These are disjoint, so there is no conflict.
- **S** has only one production.

Every non-terminal is reachable (S → A B, B → c S b) and productive (A → ε, B → ε), and the grammar is LL(1).